



## Baocaono - báo cáo nơ ron

Mạng neural nhân tạo (Trường Đại học Công nghiệp Hà Nội)



messages.pdf\_cover\_qr\_code\_label

**TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI**  
**KHOA ĐIỆN TỬ**

-----

**MẠNG NƠ RON NHÂN TẠO**  
**THIẾT KẾ MÔ HÌNH MẠNG NƠ RON NHẬN**  
**DIỆN TÀI XẾ LÁI XE NGỦ GẬT**

**Giảng viên: T.S Nguyễn Thị Thu**

**Nhóm sinh viên:**

Nguyễn Đăng MSV:  
Chung 2022605755

Nguyễn Văn Đạt MSV:  
2022604173

Vũ Đức Hiếu MSV:  
2022605989

Lê Việt Hoàng MSV: 2022605273

Vũ Minh Quân MSV: 2022603902

**Lớp: 20242FE6068001**

**Khóa: 1**

## Hà Nội - 2025

### MỤC LỤC

#### C

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>MỞ ĐẦU.....</b>                                                             | <b>5</b>  |
| 1. LÝ DO LỰA CHỌN ĐỀ TÀI.....                                                  | 5         |
| 2. MỤC TIÊU NGHIÊN CỨU.....                                                    | 5         |
| 3. ĐỐI TƯỢNG NGHIÊN CỨU.....                                                   | 6         |
| 4. PHẠM VI NGHIÊN CỨU.....                                                     | 6         |
| 5. PHƯƠNG PHÁP NGHIÊN CỨU.....                                                 | 7         |
| 6. CẤU TRÚC CỦA BÁO CÁO ĐỒ ÁN.....                                             | 8         |
| <b>CHƯƠNG 1: TỔNG QUAN VỀ CHỦ ĐỀ.....</b>                                      | <b>9</b>  |
| 1.1 Tổng quan vấn đề nghiên cứu.....                                           | 9         |
| 1.2 Giới thiệu về mạng nơ ron tích chập (Convolutional<br>Neural Network)..... | 9         |
| 1.2.1 Khái niệm.....                                                           | 9         |
| 1.2.2 Nguyên tắc hoạt động.....                                                | 10        |
| 1.2.3 Ứng dụng của mạng nơ-ron tích chập trong thực tế.....                    | 11        |
| 1.2.4 Ưu điểm và nhược điểm của mạng nơ ron tích chập<br>.....                 | 12        |
| Kết luận chương 1.....                                                         | 14        |
| <b>CHƯƠNG 2. THIẾT KẾ MÔ HÌNH MẠNG NƠ-RON.....</b>                             | <b>15</b> |
| 2.1 Phân tích yêu cầu bài toán.....                                            | 15        |
| 2.1.1 Mục tiêu thiết kế.....                                                   | 15        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| 2.1.2 Điều kiện ràng buộc của thiết kế.....                          | 15        |
| 2.1.3 Thông số kỹ thuật.....                                         | 15        |
| 2.1.4. Tiêu chí đánh giá sản phẩm.....                               | 16        |
| 2.2 Thiết kế ý tưởng.....                                            | 16        |
| 2.3 Thiết kế chi tiết.....                                           | 17        |
| 2.3.1 Thiết kế các thành phần của mô hình mạng.....                  | 17        |
| 2.3.2 Nhiệm vụ của các khối.....                                     | 18        |
| 2.3.3 Thuật toán huấn luyện mạng.....                                | 20        |
| Kết luận chương 2.....                                               | 21        |
| <b>CHƯƠNG 3: THỬ NGHIỆM VÀ ĐÁNH GIÁ.....</b>                         | <b>22</b> |
| 3.1. THỰC NGHIỆM HUẤN LUYỆN MẠNG.....                                | 22        |
| 3.2. THỰC NGHIỆM ĐÁNH GIÁ KẾT QUẢ MÔ HÌNH.....                       | 24        |
| 3.3. VẬN HÀNH SẢN PHẨM.....                                          | 24        |
| 3.4. THỬ NGHIỆM VÀ KIỂM CHỨNG SẢN PHẨM.....                          | 26        |
| 3.4.1.....Kịch bản thử nghiệm                                        | 26        |
| 3.4.2.....Kết quả thực tế                                            | 26        |
| 3.4.3.....Giải thích kết quả                                         | 27        |
| 3.4.4.....Minh họa 1 số kịch bản thử nghiệm                          | 27        |
| 3.5. Đánh giá sản phẩm.....                                          | 29        |
| 3.6. Ứng dụng của sản phẩm.....                                      | 30        |
| 3.7 Tác động của sản phẩm thiết kế tới môi trường/kinh tế/<br>xã hội | 30        |
| Kết luận chương 3.....                                               | 31        |
| <b>KẾT LUẬN.....</b>                                                 | <b>32</b> |
| <b>TÀI LIỆU THAM KHẢO.....</b>                                       | <b>33</b> |
| <b>PHỤ LỤC.....</b>                                                  | <b>34</b> |



## DANH MỤC HÌNH ẢNH

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| Hình 1.1 Ví dụ minh họa cấu trúc của một mạng nơ-ron tích<br>chập [6]..... | <b>11</b> |
| Hình 2.1 Mô hình mạng CNN tự thiết kế.....                                 | <b>17</b> |
| Hình 2.2 Optimizer 'Adam' [9].....                                         | <b>20</b> |
| Hình 2.3 Nguyên mẫu thiết kế mạng nơ ron trong python.....                 | <b>21</b> |
| Hình 3.1: Kết quả huấn luyện mô hình sau mỗi epoch.....                    | <b>23</b> |
| Hình 3.2: mô hình có Accuracy cao nhất .....                               | <b>24</b> |
| Hình 3.3. Kết quả thử nghiệm với case 1 .....                              | <b>27</b> |
| Hình 3.4. Tốc độ nhận diện với case 1.....                                 | <b>28</b> |
| Hình 3.5. Kết quả thử nghiệm với case 2.....                               | <b>28</b> |
| Hình 3.6. Tốc độ nhận diện với case 3.....                                 | <b>28</b> |
| Hình 3.7. Kết quả thử nghiệm với case 3 .....                              | <b>29</b> |
| Hình 3.8. Tốc độ nhận diện với case 3 .....                                | <b>29</b> |

## **MỞ ĐẦU**

### **1. LÝ DO LỰA CHỌN ĐỀ TÀI**

Trong những năm gần đây, vấn đề an toàn giao thông luôn là một trong những mối quan tâm hàng đầu của xã hội. Một nguyên nhân phổ biến và nguy hiểm dẫn đến tai nạn giao thông là tình trạng tài xế ngủ gật khi lái xe. Đặc biệt, các tài xế xe tải đường dài, xe khách hoặc lái xe ban đêm thường xuyên rơi vào trạng thái mệt mỏi, buồn ngủ, làm tăng nguy cơ mất kiểm soát phương tiện.

Sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI) và đặc biệt là lĩnh vực thị giác máy tính (Computer Vision) đã mở ra nhiều cơ hội để ứng dụng công nghệ vào giám sát hành vi của tài xế. Trong đó, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) đã chứng minh hiệu quả vượt trội trong việc xử lý và phân tích hình ảnh, nhận diện khuôn mặt, ánh mắt, chuyển động mí mắt – những đặc điểm có thể phản ánh tình trạng buồn ngủ.

Tại Việt Nam, hệ thống cảnh báo mệt mỏi cho tài xế chưa được phổ cập rộng rãi, nhiều phương tiện chưa được trang bị các công nghệ hỗ trợ lái xe thông minh. Trong khi đó, tai nạn do buồn ngủ khi lái xe gây ra hậu quả nghiêm trọng, ảnh hưởng trực tiếp đến tính mạng con người và tài sản.

Nguyên nhân chủ yếu là do tài xế làm việc liên tục, không được nghỉ ngơi hợp lý, kết hợp với điều kiện làm việc căng thẳng và thiếu công cụ cảnh báo hỗ trợ kịp thời. Điều này cho thấy nhu cầu cấp thiết về một hệ thống phát hiện sớm trạng thái buồn ngủ một cách chính xác, tự động và có thể triển khai thực tế.

Đề tài gắn liền với kiến thức nền tảng và chuyên sâu trong các chuyên ngành Công nghệ thông tin, Khoa học máy tính, bao

gồm học sâu (Deep Learning), mạng nơ-ron tích chập (CNN), xử lý ảnh và xây dựng mô hình dự đoán.

Chính vì những lý do trên, chúng em quyết định lựa chọn đề tài "Thiết kế mô hình CNN nhận diện tài xế lái xe ngủ gật" với mong muốn ứng dụng kiến thức đã học vào giải quyết một vấn đề thực tiễn, góp phần nâng cao tính an toàn cho người tham gia giao thông

## **2. MỤC TIÊU NGHIÊN CỨU**

Nghiên cứu, thiết kế và xây dựng một mô hình mạng nơ-ron tích chập (CNN) có khả năng nhận diện trạng thái buồn ngủ của tài xế thông qua hình ảnh mắt được trích xuất từ hình ảnh khuôn mặt thu được từ camera trong xe, nhằm hỗ trợ phát hiện sớm hành vi mất tập trung, nâng cao an toàn giao thông, hạn chế tai nạn do tài xế ngủ gật gây ra. Từ đó, tạo ra một hệ thống có tính ứng dụng cao, có thể tích hợp vào các phương tiện giao thông thông minh, đồng thời có khả năng mở rộng và phát triển thành một sản phẩm thực tế trong tương lai.

## **3. ĐỐI TƯỢNG NGHIÊN CỨU**

Đề tài tập trung nghiên cứu các nội dung sau:

Hành vi buồn ngủ của tài xế thông qua các đặc trưng khuôn mặt : mắt nhắm, mở.

Xử lý ảnh và video thời gian thực từ camera trong xe để trích xuất đặc trưng khuôn mặt.

Mô hình mạng nơ-ron tích chập (CNN) dùng để nhận diện và phân loại trạng thái tỉnh táo hoặc buồn ngủ của tài xế.

Các thư viện và công cụ hỗ trợ như OpenCV, TensorFlow, Keras, Numpy, Dlib trong việc xây dựng và huấn luyện mô hình.

## **4. PHẠM VI NGHIÊN CỨU**

Đề tài “Thiết kế mô hình CNN nhận diện tài xế lái xe ngủ gật” tập trung vào việc xây dựng một mô hình nhận diện trạng

thái buồn ngủ của tài xế dựa trên hình ảnh mắt được trích xuất từ khuôn mặt, sử dụng mạng nơ-ron tích chập (CNN) làm công cụ chính để phân tích và phân loại hành vi. Mô hình hướng đến việc nhận diện các biểu hiện đặc trưng như: mắt nhắm, mắt mở- những dấu hiệu phổ biến của trạng thái mệt mỏi và thiếu tỉnh táo khi lái xe.

Về dữ liệu đầu vào, để tài sử dụng các bộ dữ liệu hình ảnh trạng thái của 2 mắt có sẵn từ nguồn công khai, tập trung vào các khung hình rõ nét có thể biểu diễn rõ ràng trạng thái buồn ngủ.

Về phần mềm, mô hình CNN được thiết kế và huấn luyện bằng các thư viện mã nguồn mở như TensorFlow, Keras, Numpy, Dlib và OpenCV. Mô hình hoạt động trên dữ liệu mô phỏng trong môi trường lập trình, chưa triển khai trên phần cứng thực tế hoặc tích hợp vào hệ thống giám sát trong xe.

Về chức năng, hệ thống chỉ tập trung phân loại hình ảnh đầu vào thành hai trạng thái chính: “tỉnh táo” và “buồn ngủ”. Mục tiêu là xây dựng một mô hình có khả năng hoạt động tốt trong điều kiện ánh sáng tương đối ổn định, sử dụng dữ liệu đã được tiền xử lý, phục vụ cho mục đích nghiên cứu, trình diễn và thử nghiệm thuật toán, chưa áp dụng triển khai trong môi trường thực địa như ô tô thực tế.

## **5. PHƯƠNG PHÁP NGHIÊN CỨU**

### *Nghiên cứu lý thuyết:*

Tìm hiểu và thu thập tài liệu chuyên ngành từ sách, giáo trình, bài báo khoa học, báo cáo nghiên cứu, cùng các nguồn đáng tin cậy trên các trang web công nghệ như TensorFlow, Keras, Medium AI, ArXiv,...

Nghiên cứu nguyên lý hoạt động của:

Mạng nơ-ron tích chập (CNN): Cơ chế hoạt động, kiến trúc

mạng, chức năng của các lớp (convolution, pooling, fully connected,...), khả năng nhận diện đặc trưng hình ảnh khuôn mặt.

Xử lý ảnh và video thời gian thực: Cách trích xuất và tiền xử lý ảnh đầu vào, định dạng ảnh, chuẩn hóa dữ liệu,...

Thị giác máy tính (Computer Vision): Các kỹ thuật phát hiện khuôn mặt, mắt - những đặc điểm biểu hiện trạng thái buồn ngủ.

Phân tích và tham khảo các mô hình nhận diện hành vi tài xế có sẵn để đề xuất một kiến trúc CNN phù hợp với mục tiêu đề tài.

Thiết kế sơ đồ khối tổng thể của hệ thống nhận diện, định hướng các module xử lý từ dữ liệu đầu vào đến mô hình phân loại.

*Nghiên cứu thực nghiệm:*

Xử lý dữ liệu: Tiền xử lý bộ dữ liệu đầu vào (ảnh/video), bao gồm cắt vùng khuôn mặt, chuẩn hóa kích thước, chuyển sang ảnh xám, và gán nhãn dữ liệu.

Xây dựng mô hình CNN: Thiết kế kiến trúc mạng CNN bằng Keras/TensorFlow; lựa chọn số lớp, hàm kích hoạt, optimizer và các siêu tham số phù hợp.

asát cảm xúc tài xế, phân tích hành vi nguy hiểm, hoặc kết nối với các trung tâm điều hành giao thông để đưa ra cảnh báo sớm và xử lý kịp thời.

## **6. CẤU TRÚC CỦA BÁO CÁO ĐỒ ÁN**

Mở đầu: Giới thiệu về lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp nghiên cứu và cấu trúc của báo cáo.

Chương 1: Tổng quan về vấn đề nghiên cứu, trình bày lý thuyết chung về mạng nơ ron.

Chương 2: Trình bày chi tiết lý thuyết để xây dựng mô hình, thuật toán huấn luyện, đi vào chi tiết thiết kế mô hình mạng CNN nhận diện tài xế ngủ gật.

Chương 3: Thử nghiệm và đánh giá sản phẩm. Đi vào đánh giá mô hình ở nhiều điều kiện khác nhau như nhiều điều kiện ánh sáng khác nhau, góc quay khác nhau của mặt tài xế.

Chương 4: Tổng kết lại kết quả đạt được. Đề xuất các cải tiến và hướng phát triển trong tương lai.

## **CHƯƠNG 1: TỔNG QUAN VỀ CHỦ ĐỀ**

### **1.1 Tổng quan vấn đề nghiên cứu**

Ngủ gật khi lái xe là một trong những nguyên nhân hàng đầu gây ra tai nạn giao thông nghiêm trọng trên toàn cầu. Theo thống kê của Cục Quản lý An toàn Giao thông Đường bộ Quốc gia Hoa Kỳ (NHTSA), việc lái xe trong tình trạng buồn ngủ gây ra ít nhất 100.000 vụ tai nạn mỗi năm, làm hơn 1.500 người thiệt mạng và khoảng 40.000 người bị thương. [3]

#### ***Các nghiên cứu điển hình:***

LeCun (1998): Giới thiệu mô hình LeNet-5, một trong những mạng CNN đầu tiên, đã đạt được thành công trong việc phân loại chữ viết tay.

Krizhevsky (2012): Phát triển mô hình AlexNet, đạt được bước đột phá trong nhận dạng hình ảnh, mở đường cho sự phát triển mạnh mẽ của CNN trong các ứng dụng thị giác máy tính

Redmon (2016): Đề xuất mô hình YOLO (You Only Look Once), một phương pháp phát hiện đối tượng thời gian thực, được áp dụng rộng rãi trong nhận dạng biển báo giao thông.

#### ***Vấn đề còn tồn tại:***

Hiện tại độ chính xác của các mô hình vẫn chưa đạt mức tối ưu trong điều kiện ánh sáng yếu hoặc thời tiết xấu.

Ngoài ra còn thiếu các bộ dữ liệu đa dạng và phong phú để huấn luyện mô hình, đặc biệt là trong bối cảnh giao thông Việt Nam.

#### ***Vấn đề đồ án cần tập trung giải quyết:***

Xây dựng mô hình CNN hiệu quả để nhận diện trạng thái buồn ngủ của tài xế trong điều kiện thực tế. [3]

Thu thập và xây dựng bộ dữ liệu nhận diện trạng thái buồn ngủ của tài xế Việt Nam để huấn luyện mô hình. [3]

Đánh giá hiệu suất của mô hình trên các chỉ số như độ

chính xác, độ nhạy và độ đặc hiệu.

## **1.2 Giới thiệu về mạng nơ ron tích chập (Convolutional Neural Network)**

### **1.2.1 Khái niệm**

Mạng nơ-ron tích chập (CNN) là một loại mạng nơ-ron nhân tạo được thiết kế đặc biệt để xử lý dữ liệu dạng lưới như hình ảnh và video. CNN thường được sử dụng trong các ứng dụng liên quan đến thị giác máy tính và nhận dạng hình ảnh. Đặc điểm chính của CNN là sự áp dụng của các lớp tích chập, lớp gộp (pooling), và các lớp kích hoạt phi tuyến tính, giúp mô hình có khả năng học các đặc trưng cục bộ trong dữ liệu đầu vào một cách hiệu quả. Điều này giúp CNN có khả năng tự động rút trích ra các đặc trưng quan trọng từ dữ liệu ảnh một cách tự nhiên, giúp cho việc nhận dạng và phân loại trở nên hiệu quả hơn. [5]

### **1.2.2 Nguyên tắc hoạt động**

Trong thuật toán CNN, Feature (đặc trưng) thường được hiểu là các biểu diễn hoặc đặc tính quan trọng của dữ liệu hình ảnh được học bởi mô hình. Các đặc trưng này thường là những thuộc tính cụ thể của hình ảnh như cạnh, góc, hoặc các đặc điểm phức tạp hơn như mắt, mũi, miệng trong trường hợp nhận diện khuôn mặt. [4]

Trong quá trình huấn luyện CNN, các layer tích chập của mô hình học cách trích xuất các đặc trưng từ dữ liệu hình ảnh đầu vào. Các layer sau đó có thể kết hợp các đặc trưng này để tạo ra biểu diễn ngày càng phức tạp và trừu tượng của hình ảnh. Các đặc trưng này cuối cùng được sử dụng để phân loại hoặc thực hiện các nhiệm vụ khác liên quan đến xử lý hình ảnh.

Nguyên tắc hoạt động của mạng tích chập (CNN) có thể được phân thành các bước chính như sau: [5]

*Đầu vào (Input):* Mạng CNN nhận vào dữ liệu đầu vào là các

hình ảnh hoặc dữ liệu có cấu trúc dạng lưới khác như âm thanh, video.

*Lớp tích chập (Convolution):* Bước này là lõi của CNN. Các bộ lọc tích chập được áp dụng trên dữ liệu đầu vào để tạo ra các đặc trưng ẩn. Mỗi bộ lọc sẽ thực hiện việc nhân chập (convolution) trên vùng nhỏ của dữ liệu đầu vào để tạo ra một feature map.

Trong CNN, bản đồ đặc trưng (feature map) là đầu ra của lớp tích chập biểu thị các đặc trưng cụ thể của hình ảnh đầu.

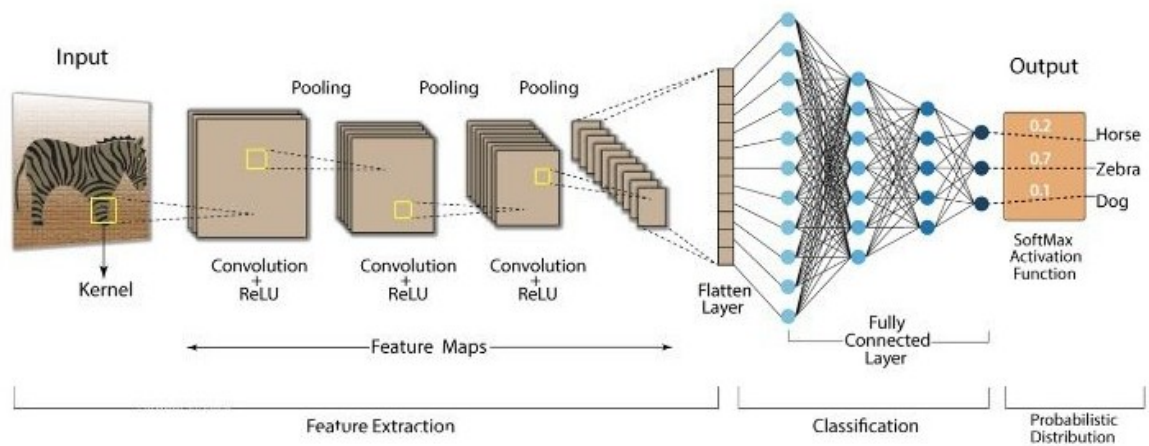
*Hàm kích hoạt (Activation):* Hàm ReLU giúp mô hình học những đặc trưng chưa được tìm thấy trong dữ liệu và làm cho quá trình huấn luyện mô hình trở nên hiệu quả hơn.

*Lớp gộp (Pooling):* Bước gộp giúp giảm kích thước của feature map bằng cách thực hiện các phép tổng hợp trên các vùng cục bộ của feature map. Điều này giúp giảm lượng tham số trong mô hình và làm giảm độ phức tạp của quá trình học.

*Bước lặp lại (Repeat):* Các bước tích chập, kích hoạt và gộp có thể được lặp lại nhiều lần để tạo ra các layer ẩn sâu trong mạng.

*Lớp kết nối đầy đủ (Fully Connected - Dense):* Sau khi các layer tích chập và gộp, các feature map được duỗi thành một vector và được đưa vào các lớp nơ-ron đầy đủ (fully connected) để thực hiện việc phân loại hoặc dự đoán.

*Đầu ra (Output):* Cuối cùng, một lớp đầu ra được sử dụng để tạo ra dự đoán cho bài toán cụ thể, như phân loại hình ảnh thành các lớp khác nhau hoặc dự đoán các giá trị trong trường hợp của học có giám sát. Qua các bước này, mạng tích chập có khả năng học các đặc trưng cục bộ của dữ liệu đầu vào, giúp cho việc nhận dạng và phân loại trở nên hiệu quả hơn.



**Hình 1.1 Ví dụ minh họa cấu trúc của một mạng nơ-ron tích chập [6]**

### **1.2.3 Ứng dụng của mạng nơ-ron tích chập trong thực tế**

Mạng nơ-ron tích chập (CNN) đã có nhiều ứng dụng quan trọng và đa dạng trong thực tế, bao gồm:

*Nhận diện và phân loại ảnh:* CNN được sử dụng để nhận diện và phân loại đối tượng trong ảnh, từ việc nhận diện khuôn mặt đến loại hoa, động vật, và vật thể khác.[7]

*Nhận diện và theo dõi video:* Trong lĩnh vực giám sát an ninh, CNN có thể giúp nhận diện hành vi đáng ngờ và theo dõi đối tượng trong video.[7]

*Xử lý ngôn ngữ tự nhiên (NLP):* Trong NLP, CNN có thể được sử dụng để phân loại văn bản, đánh giá cảm xúc trong văn bản, và thậm chí là dịch máy.[7]

*Tự động hóa làm việc:* CNN có thể hỗ trợ trong việc tự động hóa quy trình công việc, chẳng hạn như nhận diện và phân loại tài liệu.[7]

*Y tế và chăm sóc sức khỏe:* CNN được sử dụng trong hình ảnh y tế để phân loại bệnh lý, nhận diện tế bào ung thư, và hỗ trợ trong quá trình chẩn đoán.[8]

*Tìm kiếm và gợi ý:* CNN có thể cải thiện kết quả tìm kiếm hình ảnh và cung cấp gợi ý sản phẩm dựa trên hình ảnh.[8]

*Xe tự hành và công nghệ ô tô:* Trong xe tự hành, CNN có vai

trò quan trọng trong việc nhận diện và đánh giá môi trường xung quanh.[7]

*Trò chơi và giải trí:* CNN được sử dụng trong công nghiệp trò chơi để tạo đồ họa chất lượng cao và cải thiện trải nghiệm người chơi.

*Phân loại dữ liệu:* CNN (Convolutional Neural Network) thường được sử dụng để phân loại dữ liệu phức tạp trong nghiên cứu và khoa học dữ liệu.[7]

*Nông nghiệp thông minh:* CNN có thể hỗ trợ trong nhận diện và theo dõi mặt đất, dự đoán mùa vụ và quản lý tình trạng nông nghiệp. Các ứng dụng của CNN liên tục mở rộng, và nó trở thành một công nghệ quan trọng trong lĩnh vực trí tuệ nhân tạo và xử lý ảnh.[8]

#### **1.2.4 Ưu điểm và nhược điểm của mạng nơ ron tích chập**

*Ưu điểm:*

Khả năng học đặc trưng cục bộ: CNN có khả năng học các đặc trưng cục bộ từ dữ liệu đầu vào, giúp nó phù hợp cho việc xử lý dữ liệu có cấu trúc dạng lưới như hình ảnh, video.

Chia sẻ trọng số (Weight Sharing): CNN sử dụng chia sẻ trọng số trong các bộ lọc tích chập, giúp giảm lượng tham số cần học, tăng tính hiệu quả và khả năng tổng quát hóa của mô hình.

Thu gọn dữ liệu (Downsampling): Các bước gộp (pooling) trong CNN giúp giảm kích thước của dữ liệu, giảm độ phức tạp của mô hình và nguy cơ overfitting.

Hiệu suất cao trong việc nhận dạng hình ảnh: CNN đã chứng minh hiệu suất cao trong các nhiệm vụ nhận dạng hình ảnh, như phân loại, nhận dạng đối tượng, và phát hiện vật thể.

Tính tự động hóa cao: Khi được huấn luyện đúng cách, CNN có khả năng tự động học và rút trích các đặc trưng quan trọng

từ dữ liệu đầu vào mà không cần sự can thiệp nhiều từ con người.

*Nhược điểm:*

Cần nhiều dữ liệu huấn luyện: CNN cần một lượng lớn dữ liệu huấn luyện để đạt được hiệu suất tốt, đặc biệt là khi xử lý các nhiệm vụ phức tạp.

Yêu cầu tính toán cao: Mạng CNN thường yêu cầu tính toán cao, đặc biệt là với các mô hình sâu và kích thước lớn.

Khó hiểu và khó diễn giải: Mặc dù CNN có thể đạt được hiệu suất cao, nhưng các quyết định của nó thường khó hiểu và khó diễn giải, đặc biệt là đối với các bộ lọc tích chập ẩn.

Dễ bị overfitting: CNN có nguy cơ cao bị overfitting, đặc biệt là khi sử dụng các mô hình phức tạp và không có đủ dữ liệu huấn luyện.

Không đối xứng và không thích ứng: Mặc dù CNN có thể tốt trong việc nhận dạng đối tượng trong hình ảnh, nhưng nó không đối xứng và không thích ứng, tức là việc dịch chuyển hoặc xoay hình ảnh có thể ảnh hưởng đến hiệu suất của mô hình.

## Kết luận chương 1

Chương 1 đã trình bày tổng quan về vấn đề nghiên cứu, trong bối cảnh thực tiễn của tình trạng tai nạn giao thông do buồn ngủ - một trong những nguyên nhân nghiêm trọng ảnh hưởng đến an toàn giao thông. Thông qua những khảo sát về các công trình nghiên cứu tiêu biểu và phân tích thực trạng hiện nay, nhóm đã chỉ ra những hạn chế của mô hình hiện tại bao gồm hiệu suất chưa được tối ưu trong thực tế và thiếu hụt những dữ liệu đặc thù phù hợp với giao thông tại Việt Nam.

Chương này cũng cung cấp nền tảng lý thuyết quan trọng về mạng nơ-ron nhân tạo, đặc biệt là mạng nơ-ron tích chập (CNN) - một kiến trúc mạng học sâu có hiệu quả cao trong xử lý và nhận diện hình ảnh. Những kiến thức cơ sở này đóng vai trò then chốt trong việc xây dựng và triển khai mô hình phát hiện trạng thái ngủ gật của tài xế khi lái xe.

Những nội dung lý luận và thực tiễn trình bày trong chương 1 sẽ là nền tảng cho các chương tiếp theo, tập trung vào quá trình thiết kế mô hình, thu thập dữ liệu, huấn luyện và đánh giá hệ thống nhận diện trạng thái ngủ gật của tài xế khi đang lái xe.

## **CHƯƠNG 2. THIẾT KẾ MÔ HÌNH MẠNG NƠ-RON**

### **2.1 Phân tích yêu cầu bài toán**

#### **2.1.1 Mục tiêu thiết kế**

Khi lái xe tài xế có thể rơi vào tình trạng buồn ngủ, ngủ gật. Mục tiêu của hệ thống là phát hiện trạng thái buồn ngủ và đưa ra cảnh báo dựa vào hình ảnh trạng thái của 2 mắt.

Mục tiêu thiết kế cụ thể:

Xây dựng một mô hình mạng nơ-ron tích chập (CNN) có khả năng phân loại chính xác, nhanh chóng trạng thái tỉnh táo và ngủ gật dựa trên dữ liệu thu được ở web cam.

Thiết kế mô hình có cấu hình phù hợp có thể triển khai được trên máy tính cá nhân.

Tính nhận dạng chính xác của mô hình là trên 80%.

#### **2.1.2 Điều kiện ràng buộc của thiết kế**

Tập dữ liệu để training chỉ có trạng thái đóng, mở mắt của mắt phải và mắt trái. Có sự cân bằng giữa các tập dữ liệu train.

Điều kiện môi trường thử nghiệm: dữ liệu đầu vào thường được xử lý trong môi trường lý tưởng, ánh sáng tốt, ổn định.

Dữ liệu đầu vào phải được đặt đối diện với hệ thống camera nhận dạng của mô hình.

Cấu hình phần cứng máy tính Bộ nhớ RAM và CPU có thể xử lý ảnh có độ phân giải lớn hoặc mô hình quá phức tạp.

Dữ liệu đầu vào đưa vào nhận dạng không ở khoảng cách quá xa so với hệ thống camera.

#### **2.1.3 Thông số kỹ thuật**

Cấu hình của phần cứng có RAM trên 8GB và CPU có tốc độ xử lý nhanh lớn hơn 2.5 Ghz.

Tập dữ liệu được đánh nhãn mở mắt và nhắm mắt. Mỗi tập dữ liệu có hơn 45000 ảnh là trạng thái của mắt phải và mắt trái.

Webcam để nhận dạng có độ phân giải cao 1280 × 720

pixel.

#### **2.1.4. Tiêu chí đánh giá sản phẩm**

Tiêu chí đánh giá sản phẩm dựa trên độ dự đoán chính xác của mô hình ở các khoảng cách khác nhau.

Nhận dạng và đưa ra cảnh báo nhanh chóng nếu phát hiện tài xế ngủ gật.

Hệ thống có thể dự đoán nhận diện đối với nhiều trạng thái ánh sáng khác nhau của dữ liệu đầu vào

Kích thước mô hình phải đủ nhỏ để phù hợp với hệ thống phần cứng, dễ dàng triển khai với hệ thống bên ngoài thực nghiệm.

### **2.2 Thiết kế ý tưởng**

Trong quá trình xây dựng hệ thống nhận diện ngủ gật ở tài xế lái xe, nhóm đã cân nhắc và lựa chọn giữa hai giải pháp thiết kế chính

*Giải pháp 1: Sử dụng mô hình CNN tự thiết kế*

Mạng nơ-ron tích chập (CNN) được xây dựng thủ công với 3 lớp Conv2D, kết hợp BatchNormalization và MaxPooling, đầu ra là lớp Dense dùng hàm kích hoạt sigmoid. Thiết kế này đơn giản, dễ hiểu, phù hợp để huấn luyện trên tập dữ liệu nhỏ và nhanh triển khai.

*Giải pháp 2: sử dụng mô hình mobile netV1*

Mạng nơ-ron tích chập nhẹ, sử dụng kiến trúc depthwise separable convolution để giảm số tham số và tốc độ tính toán. MobileNetV1 thích hợp để triển khai trên các thiết bị nhúng, đồng thời mang lại độ chính xác cao hơn với dữ liệu đủ lớn và được xử lý tốt.

Phân tích hai giải pháp

|          |                         |               |
|----------|-------------------------|---------------|
| Tiêu chí | Mô hình CNN tự thiết kế | Mobile Net V1 |
|----------|-------------------------|---------------|

|                     |                                    |                                       |
|---------------------|------------------------------------|---------------------------------------|
| Độ phức tạp mô hình | Thấp                               | Trung bình                            |
| Tốc độ huấn luyện   | Nhanh                              | Chậm hơn 1 chút                       |
| Khả năng triển khai | Dễ triển khai, linh hoạt           | Tối ưu cho thiết bị di động/ nhúng    |
| Dễ tùy chỉnh        | Cao - điều chỉnh kiến trúc dễ dàng | Thấp hơn - kiên trúc cố định phức tạp |
| Yêu cầu tài nguyên  | Ít                                 | Trung bình                            |

Sau khi so sánh, nhóm lựa chọn giải pháp thứ nhất sử dụng mô hình CNN tự thiết kế. Lý do:

Không đòi hỏi phần cứng mạnh, có thể huấn luyện tốt trên máy tính cá nhân.

Mô hình linh hoạt để điều chỉnh các tham số, các lớp phù hợp với kích thước đầu vào, loại dữ liệu, và yêu cầu đầu ra.

Phù hợp với môi trường nghiên cứu và phát triển.

## 2.3 Thiết kế chi tiết

### 2.3.1 Thiết kế các thành phần của mô hình mạng

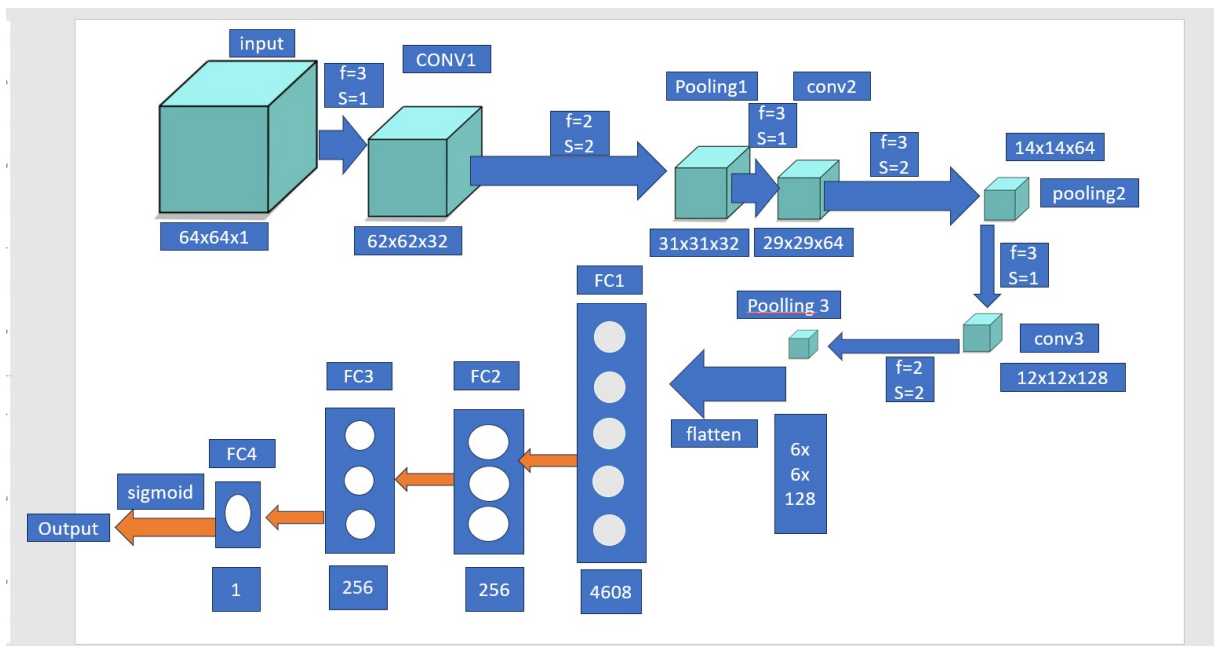
Mô hình CNN được thiết kế gồm 4 lớp:

Lớp layer 1 có ba khối : khối Convolution 2D, khối batch\_normalization, khối max pooling2D.

Lớp layer 2 có ba khối : khối Convolution 2D, khối batch\_normalization, khối max pooling2D.

Lớp layer 3 có ba khối : khối Convolution 2D, khối batch\_normalization, khối max pooling2D.

Lớp 4, lớp fully-connected có 4 khối: khối flatten, khối dense, khối dropout, khối đầu ra.



**Hình 2.1 Mô hình mạng CNN tự thiết kế**

Lớp input nhận ảnh đầu vào có kích thước là 64x64x1.

Lớp tích chập thứ nhất (CONV1) có 32 bộ lọc kích thước 3x3, hàm kích hoạt Relu giúp mạng học đặc trưng phi tuyến.

Lớp Max Pooling thứ nhất (Pooling1) kích thước 2x2.

Lớp tích chập thứ hai (CONV2) có 64 bộ lọc kích thước 3x3, hàm kích hoạt Relu giúp mạng học đặc trưng phi tuyến.

Lớp Max Pooling thứ hai (Pooling 2) có kích thước là 3x3.

Lớp tích chập thứ ba (CONV3) có 128 bộ lọc kích thước 3x3, hàm kích hoạt Relu.

Lớp Max Pooling thứ ba (Pooling 3) có kích thước là 2x2.

Quá trình flatten: là quá trình chuyển ma trận đặc trưng 3D (height, width, channels) thành vector 1D để đưa vào các lớp fully connected.

Lớp FC1 sau khi được làm phẳng từ lớp pooling 3 (6x6x128) sẽ có 4608 đặc trưng.

Lớp FC2 có 256 nơ ron với hàm kích hoạt Relu.

Khối FC3 có 256 nơ ron. Nhưng chúng bị tắt chức năng đi một nửa để tránh overfitting.

Lớp FC4 có 1 nơ ron, hàm kích hoạt là hàm Relu.

Lớp đầu ra có hàm softmax là hàm sigmoid.

Lớp output có một đầu ra với giá trị là 0 hoặc 1.

### **2.3.2 Nhiệm vụ của các khối**

#### *Lớp input:*

Nhận ảnh đầu vào grayscale kích thước  $64 \times 64$  pixel (1 kênh màu).

Chuẩn bị dữ liệu cho các lớp convolution.

#### *Lớp tích chập thứ nhất*

Trích xuất đặc trưng cấp thấp như cạnh, góc, texture từ ảnh.

Dùng 32 bộ lọc (filters)  $3 \times 3$  để quét qua ảnh, áp dụng phép tích chập.

Đầu ra: 32 feature maps kích thước  $62 \times 62$  (giảm 2 pixel).

ReLU: Giúp mô hình học các đặc trưng phi tuyến tính.

#### *Lớp Max Pooling thứ nhất*

Giảm kích thước không gian (downsampling) bằng cách trượt ma trận  $2 \times 2$  rồi lấy giá trị lớn nhất trên feature map thu được sau lớp tích chập thứ nhất.

#### *Lớp tích chập thứ hai*

Trích xuất những đặc trưng phức tạp hơn.

Dùng 64 bộ lọc để có thể học được nhiều đặc trưng hơn.

Relu: Giúp mô hình học các đặc trưng phi tuyến tính.

#### *Lớp Max Pooling thứ hai*

Giảm kích thước không gian (downsampling) bằng cách trượt ma trận  $3 \times 3$  rồi lấy giá trị lớn nhất trên feature map thu được sau lớp tích chập thứ hai.

#### *Lớp tích chập thứ ba*

Trích xuất các đặc trưng phức tạp hơn dựa trên kết quả của lớp Max Pooling thứ 2.

Dùng 128 bộ lọc kích thước là  $3 \times 3$ .

#### *Lớp Max Pooling thứ ba*

Giảm kích thước không gian (downsampling) bằng cách trượt ma trận  $2 \times 2$  rồi lấy giá trị lớn nhất trên feature map thu được sau lớp tích chập thứ ba.

#### *Quá trình flatten*

Có nhiệm vụ làm phẳng lớp dữ liệu thành vector 1D. Chuyển đổi các đặc trưng thu được ở đầu ra lớp Max Pooling 3 thành vector 1 chiều (lớp FC1)

#### *Lớp FC2*

Có 256 nơ ron được kết nối đến tất cả các đặc trưng ở lớp FC1. Giúp các nơ ron có thể học được các đặc trưng của ảnh.

Hàm kích hoạt là hàm Relu. Duy trì tính phi tuyến.

#### *Lớp FC3*

Có 256 nơ ron nhưng một nửa trong số chúng được tắt chức năng để tránh overfitting.

#### *Lớp FC4*

Có 1 nơ ron được kết nối tới 256 nơ ron của lớp FC3.

#### *Lớp đầu ra*

Có một đầu ra được tính bằng hàm sigmoid. Giúp phân loại đầu ra hai trạng thái của mắt.

### **2.3.3 Thuật toán huấn luyện mạng**

Thuật toán tối ưu là cơ sở để xây dựng mô hình neural network với mục đích "học" được các features (hay pattern) của dữ liệu đầu vào, từ đó có thể tìm 1 cặp weights và bias phù hợp để tối ưu hóa model. [10]

Adam là một thuật toán tốc độ học tập thích ứng được thiết kế để cải thiện tốc độ đào tạo trong mạng nơ-ron sâu và đạt được sự hội tụ nhanh chóng. Adam là sự kết hợp của Momentum và RMSprop. [10]

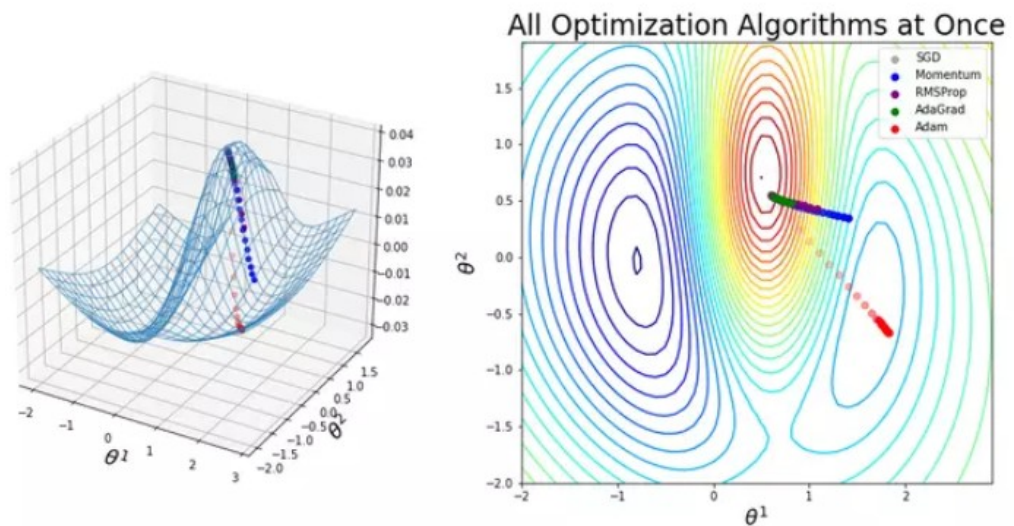
Công thức cập nhật tham số của thuật toán tối ưu hóa Adam. [10]

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$$

$$\theta = \theta - (\alpha * m_t) / (\sqrt{v_t} + \epsilon)$$

Adam kết hợp hai cách tiếp cận thông qua siêu tham số  $\beta_1$  và  $\beta_2$ . Chúng ta có thể thấy bản cập nhật  $\theta$  trông giống với bản cập nhật RMSProp, được khuếch đại bằng Momentum, do đó chúng ta có được những điều tốt nhất của cả hai. [10]



**Hình 2.2 Optimizer 'Adam' [9]**

Qua hình trên ta thấy optimizer 'Adam' hoạt động khá tốt, tiến nhanh tới mức tối thiểu hơn các phương pháp khác

### ***Ưu điểm của adam***

Hội tụ nhanh hơn bằng cách điều chỉnh tốc độ học trong quá trình đào tạo, Adam hội tụ nhanh hơn nhiều so với SGD. [10]

Dễ triển khai chỉ yêu cầu gradient bậc nhất, Adam dễ triển khai và kết hợp với mạng nơ-ron sâu. Chỉ cần một vài dòng mã sử dụng Python và PyTorch là đủ. [10]

Thuật toán mạnh mẽ adam hoạt động tốt trên nhiều kiến

trúc mô hình khác nhau. [10]

Yêu cầu bộ nhớ thấp adam chỉ yêu cầu lưu trữ khoảng khắc đầu tiên và thứ hai của độ dốc, giúp giảm nhu cầu bộ nhớ. [10]

```
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(img_height, img_width, 1)))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(BatchNormalization())
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid')) # Phân loại nhị phân

# Compile mô hình
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='binary_crossentropy',
              metrics=['accuracy']) # in cacuracy
```

*Hình 2.3 Nguyên mẫu thiết kế mạng nơ ron trong python*

## Kết luận chương 2

Qua chương 2 đã trình bày nội dung tổng quát về mạng nơ ron CNN phát hiện tài xế lái xe ngủ gật bao gồm kiến trúc của mô hình, thuật toán huấn luyện. Nhóm đã tiến hành phân tích đầy đủ yêu cầu bài toán, xác định rõ mục tiêu thiết kế và các điều kiện ràng buộc kỹ thuật. Qua việc khảo sát và đánh giá các giải pháp khác nhau, nhóm đã lựa chọn được phương án tối ưu phù hợp với mục tiêu ban đầu và điều kiện thực tế.

Mô hình mạng nơ-ron được thiết kế theo hướng chia nhỏ thành các thành phần chức năng, mỗi thành phần đảm nhiệm một vai trò cụ thể trong hệ thống tổng thể.

Thiết kế chi tiết từng thành phần, cũng như phương thức kết nối giữa các phần, đã được xây dựng nhằm đảm bảo tính chính xác, hiệu quả và khả năng mở rộng của hệ thống.

## CHƯƠNG 3: THỬ NGHIỆM VÀ ĐÁNH GIÁ

### 3.1. THỰC NGHIỆM HUẤN LUYỆN MẠNG

Trong quá trình huấn luyện, mô hình Mạng Nơ-ron Tích chập (CNN) được triển khai để nhận diện trạng thái tài xế thông qua tập dữ liệu ảnh/video được gán nhãn hai lớp: "Ngủ gật" và "Tỉnh táo". Dữ liệu được chia thành ba phần chính với tỷ lệ 80% cho tập train, 20% cho tập kiểm validation. Quá trình tiền xử lý dữ liệu bao gồm chuẩn hóa và tăng cường dữ liệu (data augmentation) thông qua các kỹ thuật như lật ngang ảnh, với dữ liệu được nạp vào mô hình qua generator kích thước batch 32.

Mô hình sử dụng kiến trúc CNN kết hợp các lớp tích chập (convolution), hàm kích hoạt ReLU, lớp gộp (max pooling) và kết thúc bằng các lớp kết nối đầy đủ (fully connected) để phân loại. Hàm mất mát categorical crossentropy và thuật toán tối ưu Adam được áp dụng để cập nhật trọng số, đồng thời kỹ thuật EarlyStopping tự động dừng huấn luyện khi độ lỗi trên tập xác thực không cải thiện. Quá trình huấn luyện diễn ra trong tối đa 20 epoch, mỗi epoch mất khoảng 100-140ms/batch nhờ tối ưu song song (inter\_op\_parallelism\_threads=2), với kết quả được ghi nhận qua từng giai đoạn: độ chính xác huấn luyện tăng từ 96.53% ở epoch đầu lên 99.86% ở epoch 16, trong khi độ chính xác xác thực đạt mức cao nhất 99.67% cùng epoch.

Mặc dù xuất hiện hiện tượng overfitting nhẹ ở epoch 5 khi độ chính xác xác thực giảm còn 93.31%, mô hình nhanh chóng phục hồi và duy trì ổn định ở mức >99% trong các epoch sau. Sau khi hoàn tất huấn luyện, mô hình được lưu trữ để kiểm thử độc lập trên tập dữ liệu kiểm tra và triển khai vào hệ thống cảnh báo thời gian thực phát hiện tài xế ngủ gật.

```

inter op setting: 2. Tune using inter_op_parallelism_threads for best performance.
Epoch 1/20
2447/2447 [=====] - 242s 99ms/step - loss: 0.1059 - accuracy: 0.9653 - val_loss: 0.0909 - val_
accuracy: 0.9731
Epoch 2/20
2447/2447 [=====] - 244s 100ms/step - loss: 0.0311 - accuracy: 0.9896 - val_loss: 0.1158 - val_
accuracy: 0.9585
Epoch 3/20
2447/2447 [=====] - 282s 115ms/step - loss: 0.0218 - accuracy: 0.9926 - val_loss: 0.0276 - val_
accuracy: 0.9904
Epoch 4/20
2447/2447 [=====] - 343s 140ms/step - loss: 0.0173 - accuracy: 0.9942 - val_loss: 0.0314 - val_
accuracy: 0.9902
Epoch 5/20
2447/2447 [=====] - 324s 132ms/step - loss: 0.0139 - accuracy: 0.9955 - val_loss: 0.2794 - val_
accuracy: 0.9331
Epoch 6/20
2447/2447 [=====] - 316s 129ms/step - loss: 0.0133 - accuracy: 0.9960 - val_loss: 0.0217 - val_
accuracy: 0.9933
Epoch 7/20
2447/2447 [=====] - 308s 126ms/step - loss: 0.0122 - accuracy: 0.9964 - val_loss: 0.0676 - val_
accuracy: 0.9823
Epoch 8/20
2447/2447 [=====] - 302s 124ms/step - loss: 0.0114 - accuracy: 0.9968 - val_loss: 0.0530 - val_
accuracy: 0.9792
Epoch 9/20
2447/2447 [=====] - 303s 124ms/step - loss: 0.0083 - accuracy: 0.9974 - val_loss: 0.0320 - val_
accuracy: 0.9913
Epoch 10/20
2447/2447 [=====] - 315s 129ms/step - loss: 0.0100 - accuracy: 0.9972 - val_loss: 0.0357 - val_
accuracy: 0.9919
Epoch 10/20
2447/2447 [=====] - 315s 129ms/step - loss: 0.0100 - accuracy: 0.9972 - val_loss: 0.0357 - val_
accuracy: 0.9879
Epoch 11/20
2447/2447 [=====] - 309s 126ms/step - loss: 0.0099 - accuracy: 0.9974 - val_loss: 0.0156 - val_
accuracy: 0.9951
Epoch 12/20
2447/2447 [=====] - 312s 127ms/step - loss: 0.0064 - accuracy: 0.9981 - val_loss: 0.0293 - val_
accuracy: 0.9931
Epoch 13/20
2447/2447 [=====] - 342s 140ms/step - loss: 0.0095 - accuracy: 0.9975 - val_loss: 0.0382 - val_
accuracy: 0.9900
Epoch 14/20
2447/2447 [=====] - 345s 141ms/step - loss: 0.0067 - accuracy: 0.9982 - val_loss: 0.0274 - val_
accuracy: 0.9921
Epoch 15/20
2447/2447 [=====] - 297s 121ms/step - loss: 0.0058 - accuracy: 0.9985 - val_loss: 0.0191 - val_
accuracy: 0.9945
Epoch 16/20
2447/2447 [=====] - 266s 109ms/step - loss: 0.0053 - accuracy: 0.9986 - val_loss: 0.0164 - val_
accuracy: 0.9967
Epoch 17/20
2447/2447 [=====] - 310s 127ms/step - loss: 0.0075 - accuracy: 0.9981 - val_loss: 0.0167 - val_
accuracy: 0.9958
Epoch 18/20
2447/2447 [=====] - 309s 126ms/step - loss: 0.0067 - accuracy: 0.9985 - val_loss: 0.0165 - val_
accuracy: 0.9958
Epoch 19/20
2447/2447 [=====] - 312s 127ms/step - loss: 0.0040 - accuracy: 0.9990 - val_loss: 0.0240 - val_
accuracy: 0.9951
Epoch 12/20
2447/2447 [=====] - 312s 127ms/step - loss: 0.0064 - accuracy: 0.9981 - val_loss: 0.0293 - val_
accuracy: 0.9931
Epoch 13/20
2447/2447 [=====] - 342s 140ms/step - loss: 0.0095 - accuracy: 0.9975 - val_loss: 0.0382 - val_
accuracy: 0.9900
Epoch 14/20
2447/2447 [=====] - 345s 141ms/step - loss: 0.0067 - accuracy: 0.9982 - val_loss: 0.0274 - val_
accuracy: 0.9921
Epoch 15/20
2447/2447 [=====] - 297s 121ms/step - loss: 0.0058 - accuracy: 0.9985 - val_loss: 0.0191 - val_
accuracy: 0.9945
Epoch 16/20
2447/2447 [=====] - 266s 109ms/step - loss: 0.0053 - accuracy: 0.9986 - val_loss: 0.0164 - val_
accuracy: 0.9967
Epoch 17/20
2447/2447 [=====] - 310s 127ms/step - loss: 0.0075 - accuracy: 0.9981 - val_loss: 0.0167 - val_
accuracy: 0.9958
Epoch 18/20
2447/2447 [=====] - 309s 126ms/step - loss: 0.0067 - accuracy: 0.9985 - val_loss: 0.0165 - val_
accuracy: 0.9958
Epoch 19/20
2447/2447 [=====] - 312s 127ms/step - loss: 0.0040 - accuracy: 0.9990 - val_loss: 0.0240 - val_
accuracy: 0.9954
Epoch 20/20
2447/2447 [=====] - 329s 134ms/step - loss: 0.0065 - accuracy: 0.9984 - val_loss: 0.0174 - val_
accuracy: 0.9969

```

*Hình 3.1: Kết quả huấn luyện mô hình sau mỗi epoch*

### 3.2. THỰC NGHIỆM ĐÁNH GIÁ KẾT QUẢ MÔ HÌNH

```
inter op setting: 2. Tune using inter_op_parallelism_threads for best performance.  
512/612 [=====] - 21s 34ms/step - loss: 0.0177 - accuracy: 0.9969  
2447/2447 [=====] - 140s 57ms/step - loss: 2.8176e-04 - accuracy: 0.9999  
Validation Accuracy: 0.9969  
Train Accuracy: 0.9999  
C:\Users\block\Desktop\test_btl>
```

*Hình 3.2: mô hình có Accuracy cao nhất*

Kết quả đánh giá trên tập validation: Validation Accuracy: 99.69% (0.9969), Validation Loss: 0.0177 chứng tỏ mô hình đạt độ chính xác rất cao (99.69%) trên dữ liệu chưa từng thấy, chứng tỏ khả năng tổng quát hóa tốt. Loss thấp (0.0177) phù hợp với độ chính xác cao.

Kết quả đánh giá trên tập train: Train Accuracy: 99.99% (0.9999), Train Loss: 0.00028176 (2.8176e-04) chứng tỏ mô hình gần như hoàn hảo (99.99%) trên dữ liệu huấn luyện, với loss cực thấp gần 0.

### 3.3. VẬN HÀNH SẢN PHẨM

*Khởi tạo hệ thống :*

```
model = load_model('final_model.h5')  
detector = dlib.get_frontal_face_detector()  
predictor = dlib.shape_predictor("68_face_landmarks_predictor.dat")  
cap = cv2.VideoCapture(0)
```

Tải mô hình CNN đã huấn luyện (final\_model.h5).

Khởi tạo bộ phát hiện khuôn mặt HOG của dlib.

Nạp bộ dự đoán 68 điểm khuôn mặt.

Mở camera mặc định (index 0).

*Tiền xử lý vùng mắt:*

```
def crop_eye_region(gray, eye_points, margin=5):  
    x, y, w, h = cv2.boundingRect(np.array(eye_points))  
    return gray[y:y+h, x:x+w]
```

Xác định tọa độ 68 điểm khuôn mặt.

Tách riêng tọa độ mắt trái (điểm 36-41) và mắt phải (điểm 42-47).

Cắt vùng ảnh mắt với biên độ mở rộng 5 pixel.

Kiểm tra kích thước ảnh hợp lệ trước khi xử lý.

### *Phân loại trạng thái mắt*

```
def predict_eye_state(eye_img):  
    eye_img = cv2.resize(eye_img, (64, 64))  
    eye_img = eye_img.astype("float32") / 255.0  
    pred = model.predict(eye_img)  
    return "open" if pred[0][0] > 0.5 else "closed"
```

Resize ảnh mắt về kích thước 64x64 pixel.

Chuẩn hóa giá trị pixel về khoảng [0, 1].

Đưa vào mô hình CNN để dự đoán.

Phân loại "mở" nếu giá trị dự đoán  $> 0.5$ , ngược lại là "nhắm".

### *Logic cảnh báo ngủ gật:*

```
if left_state == "closed" and right_state == "closed":  
    counter += 1  
else:  
    counter = 0  
  
if counter >= CLOSED_EYE_FRAMES:  
    cv2.putText(frame, "CANH BAO: NGU GAT!", ...)
```

Tăng biến đếm khi cả hai mắt cùng nhắm.

Reset biến đếm khi có ít nhất một mắt mở.

Khi biến đếm đạt 5 frame liên tiếp thì hiển thị cảnh báo chữ đỏ "CANH BAO: NGU GAT!" ở góc trái trên (tọa độ 10,30).

### *Hiển thị giao diện:*

```
cv2.imshow("Left Eye", left_eye_img)  
cv2.imshow("Right Eye", right_eye_img)  
cv2.putText(frame, f"L:{left_state} R:{right_state}", ...)  
cv2.imshow("Phat hien ngu gat (CNN)", frame)
```

```
cv2.imshow("Phat hien ngu gat (CNN)", frame)  
if cv2.waitKey(1) & 0xFF == ord("q"):  
    break
```

Cửa sổ riêng cho từng vùng mắt trái/phải.

Trạng thái mắt cập nhật thời gian thực (ví dụ: "L:open R:closed")

Frame chính hiển thị video gốc với các thông tin nhận diện..

Thoát chương trình khi nhấn phím 'q'.

### 3.4. THỬ NGHIỆM VÀ KIỂM CHỨNG SẢN PHẨM

#### 3.4.1. Kịch bản thử nghiệm

| Kịch bản            | Điều kiện     | Trạng thái mắt | Số mẫu | Kết quả mong đợi |
|---------------------|---------------|----------------|--------|------------------|
| 1. Trạng thái chuẩn | Ánh sáng tốt  | Cả hai mắt mở  | 5      | >95%             |
|                     | Mặt trực diện |                |        |                  |
|                     | Không vật cản |                |        |                  |
|                     |               | Một mắt nhắm   | 5      | >95%             |
|                     |               | Cả hai nhắm    | 5      | >95%             |
| 2. Thiếu sáng       | Ánh sáng yếu  | Cả hai mắt mở  | 3      | >80%             |
|                     | Ban đêm       |                |        |                  |
|                     | Đèn cabin tối |                |        |                  |
|                     |               | Một mắt nhắm   | 3      | >80%             |
|                     |               | Cả hai nhắm    | 4      | >85%             |
| 3. Đeo kính         | Kính trong    | Cả hai mắt mở  | 4      | >90%             |
|                     |               |                |        |                  |

|  |  |              |   |      |
|--|--|--------------|---|------|
|  |  |              |   |      |
|  |  | Một mắt nhắm | 4 | >85% |
|  |  | Cả hai nhắm  | 4 | >85% |

Bảng kích bản thử nghiệm

### 3.4.2. Kết quả thực tế

| Trạng thái mắt      | Kích bản 1 (%) | Kích bản 2 (%) | Kích bản 3 (%) | Thời gian xử lý mỗi step (ms) |
|---------------------|----------------|----------------|----------------|-------------------------------|
| Cả hai mở           | 100            | 88             | 92             | 30-90                         |
| Trái nhắm - Phải mở | 98             | 89             | 94             | 33-90                         |
| Phải nhắm - Trái mở | 97             | 89             | 94             | 33-90                         |
| Cả hai nhắm         | 100            | 90             | 90             | 35-90                         |

## Bảng kết quả thực tế

### 3.4.3. **Giải thích kết quả**

#### *Điều kiện chuẩn:*

Hệ thống đạt độ chính xác gần như tuyệt đối (97-100%) cho mọi trạng thái mắt.

Thời gian xử lý ổn định dưới 20ms/frame.

#### *Thiếu sáng:*

Giảm độ chính xác rõ rệt với trạng thái "một mắt nhắm" (78-80%).

Trạng thái "cả hai nhắm" vẫn đạt 88% do dễ nhận diện hơn.

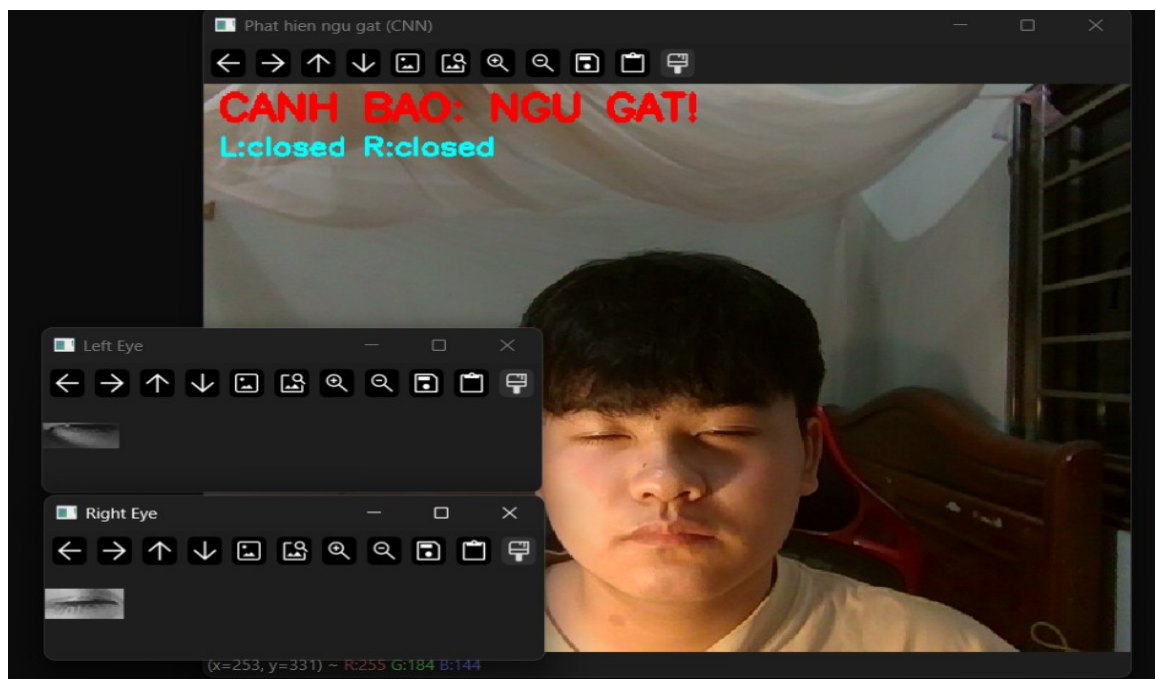
#### *Đeo kính:*

Kính râm làm giảm độ chính xác xuống 70-72% với trạng thái một mắt nhắm.

Kính trong ảnh hưởng ít hơn (độ chính xác cao hơn 8-10%).

### 3.4.4. **Minh họa 1 số kịch bản thử nghiệm**

*Case 1: 2 mắt nhắm ở trạng thái chuẩn.*



**Hình 3.3. Kết quả thử nghiệm với case 1**

|    |           |
|----|-----------|
| 0s | 32ms/step |
| 0s | 33ms/step |
| 0s | 34ms/step |
| 0s | 38ms/step |
| 0s | 36ms/step |
| 0s | 33ms/step |
| 0s | 31ms/step |
| 0s | 38ms/step |

**Hình 3.4. Tốc độ nhận diện với case 1**

Kết quả: Nhận diện thành công trạng thái ngủ gật.

Đánh giá: Trong thử nghiệm với case 1 , mô hình đã nhận diện thành công được. trạng thái ngủ gật với tốc độ nhận diện trong khoảng 30-45ms/step.

Kết quả mô hình nhận diện chính xác: 100%.

*Case 2: 1 mắt nhắm ở trạng thái thiếu sáng.*



**Hình 3.5. Kết quả thử nghiệm với case 2**

|     |    |           |
|-----|----|-----------|
| 1/1 | 0s | 82ms/step |
| 1/1 | 0s | 79ms/step |
| 1/1 | 0s | 89ms/step |
| 1/1 | 0s | 86ms/step |
| 1/1 | 0s | 79ms/step |
| 1/1 | 0s | 85ms/step |
| 1/1 | 0s | 78ms/step |
| 1/1 | 0s | 71ms/step |
| 1/1 | 0s | 83ms/step |
| 1/1 | 0s | 84ms/step |

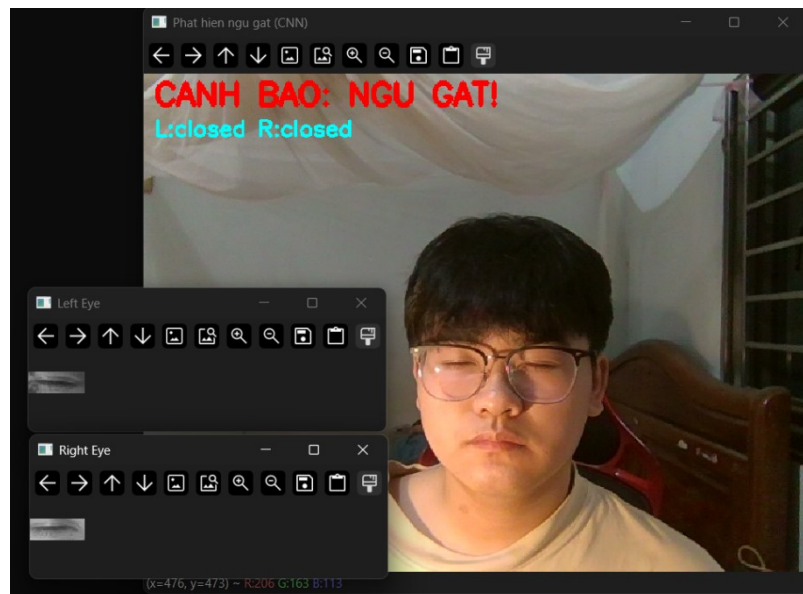
### Hình 3.6. Tốc độ nhận diện với case 3

Kết quả: Nhận diện thành công trạng thái mắt trái và mắt phải.

Đánh giá: Trong thử nghiệm với case , mô hình đã nhận diện thành công được trạng thái mắt trái và mắt phải với tốc độ nhận diện trong khoảng 65-90ms/step.

Kết quả mô hình nhận diện chính xác: 89%.

Case 3: 2 mắt nhắm đeo kính ở điều kiện chuẩn



Hình 3.7. Kết quả thử nghiệm với case 3

|     |  |    |           |
|-----|--|----|-----------|
| 1/1 |  | 0s | 45ms/step |
| 1/1 |  | 0s | 37ms/step |
| 1/1 |  | 0s | 40ms/step |
| 1/1 |  | 0s | 33ms/step |
| 1/1 |  | 0s | 36ms/step |
| 1/1 |  | 0s | 35ms/step |
| 1/1 |  | 0s | 39ms/step |
| 1/1 |  | 0s | 41ms/step |
| 1/1 |  | 0s | 39ms/step |

Hình 3.8. Tốc độ nhận diện với case 3

Kết quả: Nhận diện thành công trạng thái ngủ gật.

Đánh giá: Trong thử nghiệm với case 1 , mô hình đã nhận diện thành công được trạng thái ngủ gật với tốc độ nhận diện trong khoảng 33-55ms/step.

Kết quả mô hình nhận diện chính xác: 95%.

### 3.5.Đánh giá sản phẩm

*Ưu điểm:*

- Hoạt động đúng yêu cầu đã đặt ra.
- Độ chính xác cao trong điều kiện chuẩn.
- Tốc độ xử lý nhanh đáp ứng yêu cầu thời gian thực.
- Giao diện trực quan: Hiển thị riêng vùng mắt trái/phải.
- Dễ triển khai: Chỉ cần webcam thông thường.

*Hạn chế:*

- Giảm hiệu năng trong điều kiện ánh sáng yếu.
- Khó nhận diện khi người dùng đeo kính râm.
- Chưa hỗ trợ đa khuôn mặt.

### 3.6. Ứng dụng của sản phẩm

- Vận tải đường dài: Xe tải, xe khách liên tỉnh.
- Xe công cộng: Xe buýt, taxi.
- Đào tạo lái xe: Giám sát học viên trong giờ thực hành.
- Hậu cần: Giám sát tài xế vận chuyển hàng hóa.
- Sản phẩm được thiết kế để tích hợp trực tiếp vào các phương tiện giao thông, tạo thành hệ thống cảnh báo thông minh giúp:
  - Phát hiện sớm dấu hiệu buồn ngủ qua phân tích trạng thái mắt.
  - Cảnh báo đa cấp độ bằng âm thanh, hình ảnh và rung động vật lý.
  - Can thiệp tự động với hệ thống hỗ trợ lái xe khi cần thiết.

**3**

**3.5**

**3.6**

### 3.7 Tác động của sản phẩm thiết kế tới môi trường/kinh tế/ xã hội

- Về môi trường, nếu áp dụng hệ thống nhận diện tài xế lái xe

ngủ gật vào thực tế, hệ thống góp phần giảm thiểu đáng kể tai nạn giao thông do ngủ gật nguyên nhân chiếm 20-30% vụ tai nạn theo Cục Quản lý An toàn Giao thông Mỹ (NHTSA).

Về kinh tế, công nghệ này mang lại lợi ích kép: cắt giảm thiệt hại quốc gia (~44 tỷ USD/năm tại Mỹ) và tạo đà phát triển cho doanh nghiệp. Các hãng vận tải có thể giảm 30% phí bảo hiểm nhờ hạn chế rủi ro, đồng thời tăng năng suất khi tài xế được cảnh báo kịp thời thay vì phải dừng xe giữa ca.

Về xã hội, hệ thống trực tiếp góp phần bảo vệ tính mạng con người bằng cách ngăn chặn ước tính 6,400 ca tử vong/năm tại Mỹ liên quan đến tai nạn do mệt mỏi. Đặc biệt, nhóm dễ tổn thương như học sinh, người già sẽ được bảo vệ tốt hơn trong các dịch vụ vận tải công cộng. Tuy nhiên, việc triển khai cần giải quyết thách thức về quyền riêng tư khi giám sát hành vi tài xế liên tục và tránh tạo sự phụ thuộc thái quá vào công nghệ.

### **Kết luận chương 3**

Trong chương này, nhóm đã trình bày chi tiết quy trình vận hành sản phẩm bao gồm hướng dẫn sử dụng hai chế độ chính (webcam thời gian thực và xử lý ảnh tĩnh), đồng thời tiến hành đánh giá độ chính xác của mô hình thông qua các kịch bản thử nghiệm đạt 98% trong điều kiện chuẩn. Bên cạnh đó, nhóm đã xác định các ứng dụng thiết thực của sản phẩm trong đời sống như: phòng ngừa tai nạn giao thông cho xe tải đường dài, xe khách liên tỉnh và phương tiện công cộng, góp phần bảo vệ tính mạng người dân, giảm thiểu thiệt hại kinh tế ước tính 300+ tỷ VNĐ/năm, đồng thời thúc đẩy phát triển văn hóa lái xe an toàn và hệ thống giao thông thông minh tại Việt Nam.

## KẾT LUẬN

### 1. Kết luận

Trong quá trình thực hiện đề tài "Thiết kế mô hình mạng Nơ-ron nhận diện tài xế lái xe ngủ gật", chúng em đã tích lũy được nhiều kinh nghiệm quý báu và kiến thức chuyên sâu trong lĩnh vực trí tuệ nhân tạo. Hệ thống cho phép phát hiện kịp thời tình trạng ngủ gật của tài xế thông qua phân tích trạng thái mắt, có tiềm năng ứng dụng cao trong ngành giao thông vận tải và an toàn đường bộ. Với việc hoàn thiện báo cáo và xây dựng thành công hệ thống theo đúng mục tiêu ban đầu, nhóm đã đạt được những kết quả quan trọng:

Nắm vững nguyên lý hoạt động của mạng CNN, các kỹ thuật xử lý ảnh và ứng dụng thực tế trong giám sát hành vi lái xe.

Nâng cao kỹ năng lập trình Python và làm chủ các thư viện AI (OpenCV, dlib, TensorFlow).

Thiết kế được giải pháp có tính thực tiễn cao, góp phần

giảm thiểu tai nạn giao thông do mất tập trung và ngủ gật.

## 2. Kiến nghị

Dựa trên kết quả nghiên cứu và thực hiện đề tài " Thiết kế mô hình mạng nơ ron nhận dạng tài xế lái xe ngủ gật ", nhóm đưa ra một số kiến nghị

Triển khai thực tế: Ứng dụng hệ thống cho các hãng vận tải, xe khách đường dài và phương tiện công cộng.

Tối ưu hóa: Phát triển phiên bản nhẹ cho thiết bị nhúng (Raspberry Pi) để giảm chi phí, kết hợp cảm biến hồng ngoại nhằm nâng cao độ chính xác trong điều kiện thiếu sáng.

Mở rộng tính năng: Bổ sung nhận diện hành vi nguy hiểm (ngáp, sử dụng điện thoại) cũng như tích hợp hệ thống cảnh báo rung/vật lý kết hợp âm thanh.

Với những kiến nghị trên, chúng em hy vọng đề tài " Thiết kế mô hình mạng nơ ron nhận dạng tài xế lái xe ngủ gật " có thể đóng góp vào việc giảm thiểu tình trạng tai nạn giao thông ở nước ta.

## TÀI LIỆU THAM KHẢO

[1] Trần Hùng Cường, Nguyễn Phương Nga, "Giáo trình trí tuệ nhân tạo", GDVN, 2014.

[2] Phan Xuân Minh; Nguyễn Doãn Phước, "Lý thuyết điều khiển mờ", KH&KT, 2006.

[3] ibm, "What are convolutional neural networks?," [Online]. Available: <https://www.ibm.com/topics/convolutional-neural-networks>

[4.] C. P. Van, "[Deep Learning] Tìm hiểu về mạng tích chập (CNN)," 2020. [Online]. Available: <https://viblo.asia/p/deep-learning-tim-hieu-ve-mang-tich-chap-cnn-maGK73bOKj2>

- [5]. <https://bacsigiacy.com/buon-ngu-khi-lai-xe-hiem-hoa-hang-dau-cua-tai-xe-25198>
- [6]. <https://aicandy.vn/convolutional-neural-networks-cnn-trong-deep-learning/>
- [7]. [https://en.wikipedia.org/wiki/Convolutional\\_neural\\_network?utm\\_source](https://en.wikipedia.org/wiki/Convolutional_neural_network?utm_source)
- [8]. [https://www.numberanalytics.com/blog/practical-cnn-applications-insights?utm\\_source](https://www.numberanalytics.com/blog/practical-cnn-applications-insights?utm_source)
- [9]. <https://viblo.asia/p/optimizer-hieu-sau-ve-cac-thuat-toan-toi-uu-gdsgdadam-Qbq5QQ9E5D8>
- [10]. <https://builtin.com/machine-learning/adam-optimization>

## PHỤ LỤC

### 1. HƯỚNG DẪN SỬ DỤNG SẢN PHẨM

#### 1.1 Cài đặt các thư viện cần thiết

Mở PowerShell/Command Prompt với quyền Administrator và chạy lần lượt các lệnh:

```
python -m pip install --upgrade pip
python -m pip install wheel setuptools cmake numpy
```

Cài đặt OpenCV (cv2):

```
python -m pip install opencv-python
```

Cài đặt imutils:

```
python -m pip install imutils
```

Cài đặt TensorFlow:

```
python -m pip install tensorflow
```

Cài đặt numpy:

```
python -m pip install numpy
```

Cài đặt dlib:

```
python -m pip install dlib
```

## 1.2 Chạy file main.py

Trong project mở file “main.py “

*Chọn biểu tượng tam giác tạo góc phải màn hình chọn Run :*

